

Module 02 — Creational Patterns I: Factories

Contents

1	Learning outcomes	2
2	Motivation	2
3	Singleton	2
3.1	Problem	2
3.2	Intent	2
3.3	Structure	3
3.4	Classic Java	3
3.5	Modern Java 27	4
3.6	Why Singleton is often an anti-pattern	4
3.7	Sidebar: Lazy Constants (preview)	4
3.8	Real-world usage	5
3.9	Pitfalls and when NOT to use it	5
3.10	Related patterns	5
4	Static Factory Method	5
4.1	Problem	5
4.2	Intent	5
4.3	Structure	5
4.4	Classic Java	5
4.5	Modern Java 27	6
4.6	Real-world usage	7
4.7	Pitfalls and when NOT to use it	7
4.8	Related patterns	7
5	Factory Method	7
5.1	Problem	7
5.2	Intent	7
5.3	Structure	8
5.4	Classic Java	8
5.5	Modern Java 27	9
5.6	A second example: logistics	9
5.7	Real-world usage	9
5.8	Pitfalls and when NOT to use it	9
5.9	Related patterns	9
6	Abstract Factory	9
6.1	Problem	9
6.2	Intent	9
6.3	Structure	10
6.4	Classic Java	10
6.5	Modern Java 27	10
6.6	A second example: cloud providers	11
6.7	Real-world usage	11
6.8	Pitfalls and when NOT to use it	11
6.9	Related patterns	11

7	ServiceLoader	12
7.1	Problem	12
7.2	How it works	12
7.3	Example	12
7.4	Real-world usage	13
7.5	Pitfalls and when NOT to use it	13
8	Choosing a creational technique	13
9	Summary	13
10	Quiz	14
11	Assignments	14
12	Further reading	14

Week 3 · Prerequisites: m01 (OCP, DIP, composition root) · Estimated study time: 5 h

```
Run          every          example          without          a          build:
java modules/m02-creational-factories/src/main/java/io/github/aliturgutbozkurt/ (JDK
patterns/m02/examples/<path>/<Demo>.java
27)
```

1 Learning outcomes

By the end of this module you can:

1. **Implement** a thread-safe Singleton (enum, lazy holder) and **explain** why global state hurts testability.
2. **Write** static factory methods with meaningful names, instance caching and subtype selection.
3. **Implement** Factory Method in its classic (subclass hook) and modern (`Supplier`, enum registry) forms.
4. **Implement** an Abstract Factory that keeps a family of products consistent.
5. **Load** implementations at run time with `ServiceLoader`.
6. **Decide** which creational technique fits a problem — including "just call the constructor".

2 Motivation

`new HtmlFormatter()` looks harmless. But the line that says `new` decides *which class* is used, and m01 taught us that such decisions belong in as few places as possible (OCP, DIP). The five techniques in this module move that decision somewhere better: into a named method, a subclass, a family object, or a configuration file.

3 Singleton

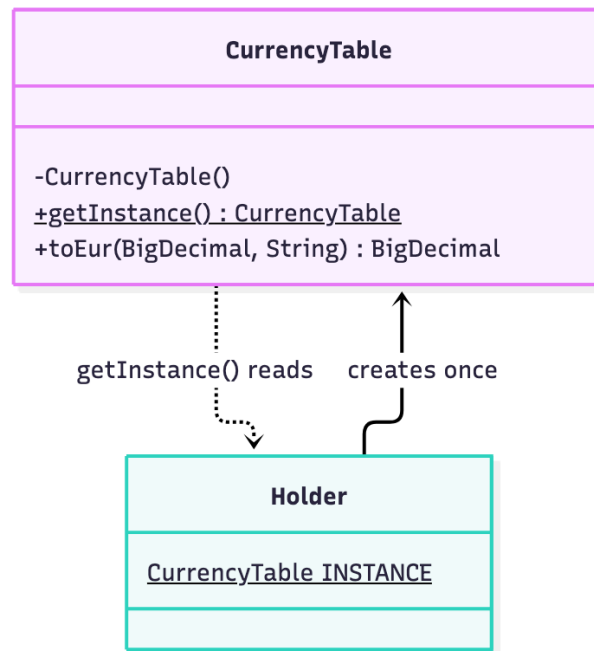
3.1 Problem

Some things should exist once per application: a settings object, a table of exchange rates loaded from a slow service. Creating them twice wastes time or, worse, lets two copies disagree.

3.2 Intent

Ensure a class has only **one instance** and provide a global point of access to it.

3.3 Structure



3.4 Classic Java

A private constructor stops everyone else from calling `new`; a static field holds the one instance:

```
// file: examples/singleton/testability/before/SequenceGenerator.java
public final class SequenceGenerator {

    private static final SequenceGenerator INSTANCE = new SequenceGenerator();

    // Mutable state behind a global access point – the problem this example
    // demonstrates.
    private int next = 1;

    private SequenceGenerator() {}

    public static SequenceGenerator getInstance() {
        return INSTANCE;
    }
}
```

This instance is created when the class is initialized. If creation is expensive, the **lazy holder idiom** delays it until the first `getInstance()` call. The JVM initializes the nested `Holder` class only when it is first used, and class initialization is thread-safe by the language specification — no `synchronized`, no double-checked locking:

```
// file: examples/singleton/holder/CurrencyTable.java
private static final class Holder {
    static final CurrencyTable INSTANCE = new CurrencyTable();
}

public static CurrencyTable getInstance() {
    return Holder.INSTANCE;
}
```

The tests prove both claims: in a fresh class loader `creations()` is 0 until `getInstance()` runs, and 1 000 virtual threads calling `getInstance()` at once see a single instance, created once.

3.5 Modern Java 27

A one-element `enum` is the simplest correct Singleton. The JVM guarantees one instance, thread-safe creation, and that neither serialization nor reflection can produce a second one (both are tested in `EnumSingletonTest`):

```
// file: examples/singleton/enumsingleton/AppSettings.java
public enum AppSettings {
    INSTANCE;

    private final Map<String, String> values = Map.of(
        "app.name", "PatternShop",
        "currency", "EUR",
        "page.size", "20");
```

3.6 Why Singleton is often an anti-pattern

Look at what `OrderService` depends on:

```
// file: examples/singleton/testability/before/OrderService.java
public String placeOrder() {
    return "ORD-" + SequenceGenerator.getInstance().next();
}
```

Nothing in its constructor or signature reveals the dependency, and the counter is **global mutable state**: two "independent" services continue each other's numbers, and a test's result depends on which tests ran before it. The fix is m01's Dependency Inversion — ask for the dependency:

```
// file: examples/singleton/testability/after/OrderService.java
public OrderService(IdSource ids) {
    this.ids = Objects.requireNonNull(ids, "ids");
}
```

"Only one instance" becomes a decision of the composition root, not a property of the class:

```
== before: OrderService calls SequenceGenerator.getInstance() ==
a second OrderService continues the first one's numbers: true
== after: the IdSource is injected ==
shared on purpose: ORD-1, ORD-2
separate sources: ORD-1, ORD-1
fixed for a test: ORD-42
```

3.7 Sidebar: Lazy Constants (preview)

 **Preview feature — not used in graded code.** JDK 27 contains *Lazy Constants* (JEP 531, third preview): a JDK-supported way to declare a value that is computed once, on first access, and then treated as a constant by the JVM. It aims to replace hand-written lazy holders. Because it is a preview API it may still change; this course uses the holder idiom. Details: [JEP 531](#) and [docs/java27-features.md](#).

3.8 Real-world usage

`Runtime.getRuntime()` is a classic Singleton. `Collections.emptyList()` returns one shared immutable instance. Frameworks such as Spring create "singleton-scoped" beans — one instance per application, but injected, not fetched through a global.

3.9 Pitfalls and when NOT to use it

- Global mutable state: hidden coupling and order-dependent tests (see above). Prefer injection.
- "One instance" is really "one instance **per class loader**" — the laziness test uses exactly this to get a fresh copy.
- Hand-rolled lazy initialization with double-checked locking is easy to get wrong (it needs `volatile`); use the holder idiom or an `enum`.

3.10 Related patterns

A `getInstance()` method is a **Static Factory Method**. Abstract factories are often single instances. m03 shows DI as the general replacement.

4 Static Factory Method

4.1 Problem

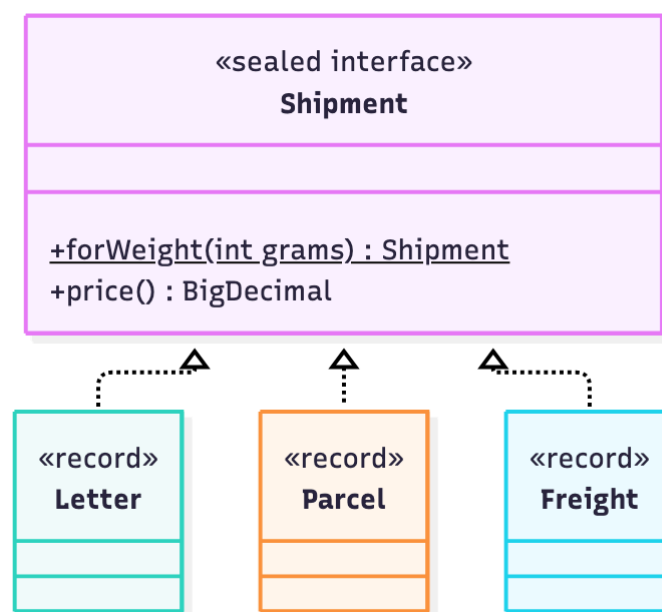
Constructors all have the class's name, cannot return an existing object, and always return exactly their own class. Those three limits become visible quickly: `new Temperature(100)` — Celsius or Fahrenheit?

4.2 Intent

Provide a **static method** that returns an instance, instead of (or next to) a public constructor.

(This is not one of the 23 GoF patterns, but it is the most common creational technique in the JDK.)

4.3 Structure



4.4 Classic Java

Names. A factory says what it does:

```
// file: examples/staticfactory/Money.java
/** {@code Money.of("12.50", "EUR")}. */
public static Money of(String amount, String currencyCode) {
    return new Money(new BigDecimal(amount), Currency.getInstance(currencyCode));
}

/** Zero in the given currency. */
public static Money zero(String currencyCode) {
    return of("0", currencyCode);
}
```

Names even let two factories share a parameter list — impossible with overloaded constructors:

```
// file: examples/staticfactory/Temperature.java
public static Temperature ofKelvin(double kelvin) {
    return new Temperature(kelvin);
}

public static Temperature ofCelsius(double celsius) {
    return new Temperature(celsius + ZERO_CELSIUS_IN_KELVIN);
}
```

Caching. `new` always creates an object; a factory may hand out a shared one, as `Integer.valueOf` does:

```
// file: examples/staticfactory/Percentage.java
/** The shared instance for {@code 0..100}; equal percentages are the same object.
 */
public static Percentage of(int value) {
    if (value < 0 || value > 100) {
        throw new IllegalArgumentException("percentage must be in 0..100: " +
value);
    }
    return CACHE[value];
}
```

4.5 Modern Java 27

Choosing a subtype. A factory on a sealed interface returns the record that fits the input; the caller just works with `Shipment`:

```
// file: examples/staticfactory/Shipment.java
/** Letter up to 500 g, parcel up to 30 kg, freight above. */
static Shipment forWeight(int grams) {
    if (grams <= 0) {
        throw new IllegalArgumentException("weight must be positive: " + grams + "
g");
    }
    if (grams <= LETTER_LIMIT_GRAMS) {
        return new Letter(grams);
    }
    return grams <= PARCEL_LIMIT_GRAMS ? new Parcel(grams) : new Freight(grams);
}
```

```
300 g -> Letter 2.50
2500 g -> Parcel 7.50
45000 g -> Freight 53.50
```

Naming conventions used across the JDK:

Name	Meaning	JDK example
<code>of</code>	build from components	<code>List.of</code> , <code>LocalDate.of</code> , <code>Path.of</code>
<code>from</code>	convert from another type	<code>Instant.from(temporal)</code>
<code>valueOf</code>	like <code>of</code> , older style, often cached	<code>Integer.valueOf</code> , <code>String.valueOf</code>
<code>parse</code>	read from text	<code>Integer.parseInt</code> , <code>Duration.parse</code>
<code>getInstance</code> / <code>instance</code>	may return a shared instance	<code>Currency.getInstance</code>
<code>newX</code>	always a new object	<code>Files.newBufferedReader</code>

4.6 Real-world usage

`List.of(...)` returns different hidden classes depending on the number of elements; `EnumSet.of(...)` picks a long-based or array-based implementation from the enum's size; `Optional.of`, `Duration.ofMinutes`, `Path.of`.

4.7 Pitfalls and when NOT to use it

- With only a private constructor the class cannot be subclassed (often a feature).
- Factories are harder to spot in the Javadoc than constructors — follow the naming conventions.
- Only cache **immutable** objects; a shared mutable instance is a Singleton in disguise.

4.8 Related patterns

Singleton's `getInstance()`; **Flyweight** (m05) is caching taken further; **Builder** (m03) for many parameters.

5 Factory Method

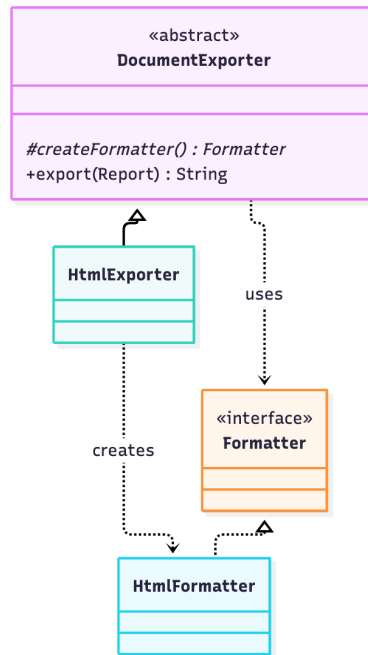
5.1 Problem

An exporter has one algorithm — write the title, the header, every row, the end — but the *formatter* it needs depends on the output format. Putting an `if (format == ...)` into the algorithm violates OCP.

5.2 Intent

Define an interface for creating an object, but let **subclasses decide which class to instantiate**.

5.3 Structure



5.4 Classic Java

The creator's algorithm is written once and calls the abstract **factory method**:

```
// file: examples/factorymethod/export/classic/DocumentExporter.java
public abstract class DocumentExporter {

    /** The factory method. */
    protected abstract Formatter createFormatter();

    public final String export(Report report) {
        Formatter formatter = createFormatter();
        var out = new StringBuilder(formatter.begin(report.title()));
        out.append(formatter.header(report.header()));
        for (List<String> row : report.rows()) {
            out.append(formatter.row(row));
        }
        return out.append(formatter.end()).toString();
    }
}
```

Each subclass only picks the product:

```
// file: examples/factorymethod/export/classic/HtmlExporter.java
public final class HtmlExporter extends DocumentExporter {

    @Override
    protected Formatter createFormatter() {
        return new HtmlFormatter();
    }
}
```

5.5 Modern Java 27

A subclass whose only job is to call a constructor is a lot of ceremony. A `Supplier<Formatter>` does the same job, and an enum can hold one constructor reference per format:

```
// file: examples/factorymethod/export/modern/ExportFormat.java
public enum ExportFormat {
    MARKDOWN(MarkdownFormatter::new),
    HTML(HtmlFormatter::new),
    CSV(CsvFormatter::new);
}
```

`ExportTest` proves the classic and the modern versions produce identical text for every format.

5.6 A second example: logistics

`Logistics.planDelivery` is business logic written once against `Transport`; `RoadLogistics` creates a `Truck`, `SeaLogistics` a `Ship`. A test adds an air mode without touching `Logistics`:

```
// file: examples/factorymethod/logistics/Logistics.java
/** The factory method. */
protected abstract Transport createTransport();

public final String planDelivery(Cargo cargo) {
    Transport transport = createTransport();
}
```

```
Road: Machine parts by Truck, 1200 km, cost 1440.00, 2 day(s)
Sea: Machine parts by Ship, 1200 km, cost 980.00, 5 day(s)
```

5.7 Real-world usage

`Iterable.iterator()` is the JDK's best-known factory method: every collection creates the iterator that fits it, and the enhanced `for` loop only knows `Iterator`. `NumberFormat.getInstance(locale)` and `Charset.newEncoder()` are others.

5.8 Pitfalls and when NOT to use it

- One subclass per product can explode; prefer a `Supplier` or an enum registry when the subclass does nothing else.
- With a single product and no variation expected, just call the constructor.

5.9 Related patterns

`export` is a **Template Method** (m06) whose one step is a factory method. An **Abstract Factory** is a set of factory methods for a whole family.

6 Abstract Factory

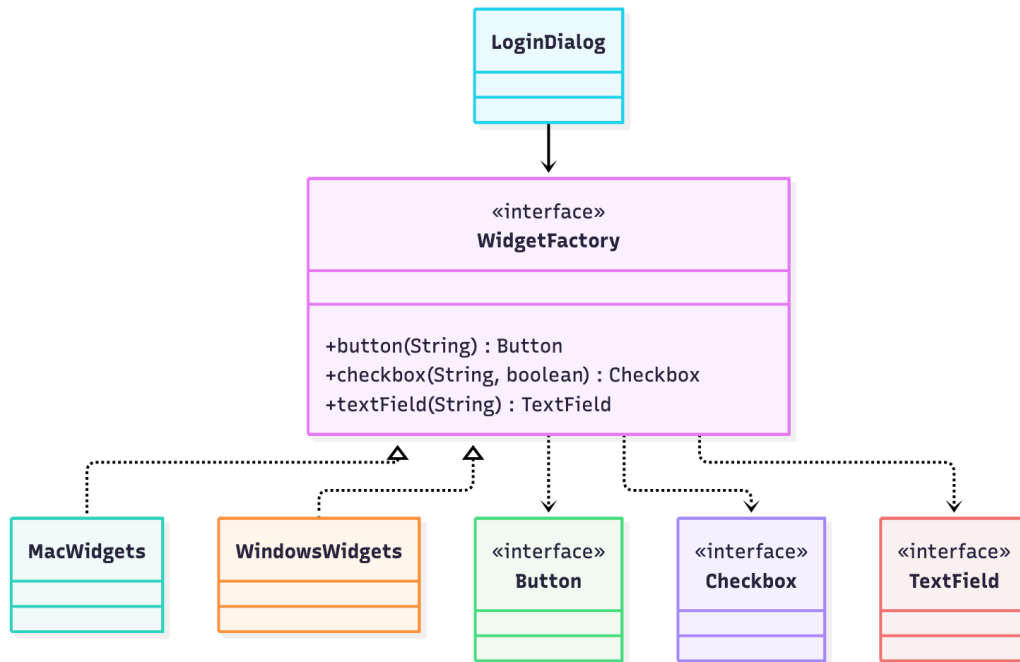
6.1 Problem

A login dialog needs a text field, a checkbox and a button. On macOS they must all look like macOS, on Windows like Windows. If the dialog calls constructors directly, one forgotten `new` gives a Windows checkbox on a Mac.

6.2 Intent

Provide an interface for creating **families of related objects** without specifying their concrete classes.

6.3 Structure



6.4 Classic Java

One creation method per product of the family:

```
// file: examples/abstractfactory/ui/WidgetFactory.java
public interface WidgetFactory {

    Button button(String label);

    Checkbox checkbox(String label, boolean checked);

    TextField textfield(String label);
}
```

The client builds the whole dialog from whatever family it receives and never names a concrete widget:

```
// file: examples/abstractfactory/ui/LoginDialog.java
public LoginDialog(WidgetFactory factory) {
    Objects.requireNonNull(factory, "factory");
    widgets = List.of(
        factory.textfield("Username"),
        factory.textfield("Password"),
        factory.checkbox("Remember me", true),
        factory.button("Log in"));
}
```

6.5 Modern Java 27

The concrete products can be **private nested records** inside their factory — clients cannot even name them, so mixing families is impossible:

```
// file: examples/abstractfactory/ui/MacWidgets.java
public final class MacWidgets implements WidgetFactory {
```

```
private static final String PLATFORM = "macOS";

private record MacButton(String label) implements Button {
    @Override public String platform() { return PLATFORM; }
    @Override public String render() { return "(" + label + " )"; }
}
```

The family is chosen **once**, in the composition root, by a static factory:

```
// file: examples/abstractfactory/ui/WidgetFactories.java
public static WidgetFactory forOs(String osName) {
    return osName.toLowerCase(Locale.ROOT).startsWith("mac") ? new MacWidgets() :
    new WindowsWidgets();
}
```

```
-- Mac OS X
[macOS] Username: (_____)
[macOS] Password: (_____)
[macOS] ● Remember me
[macOS] ( Log in )
-- Windows 11
[Windows] Username: [_____]
[Windows] Password: [_____]
[Windows] [x] Remember me
[Windows] [ Log in ]
```

6.6 A second example: cloud providers

Storage and a message queue from two fictional providers: an `acme://...` URI must never be handed to the Nimbus queue. `ReportArchiver` takes both products from one `CloudFactory`:

```
// file: examples/abstractfactory/cloud/ReportArchiver.java
public ReportArchiver(CloudFactory cloud) {
    Objects.requireNonNull(cloud, "cloud");
    this.storage = cloud.storage();
    this.queue = cloud.queue();
}
```

6.7 Real-world usage

A JDBC `Connection` is a factory for a family bound to one database: `createStatement()`, `prepareStatement(...)`, `createBlob()`. `javax.xml.parsers.DocumentBuilderFactory` and Swing's look-and-feel (`UIManager`) are others.

6.8 Pitfalls and when NOT to use it

- Adding a new **product** (say, a slider) changes the interface and every family — the expression problem from m01 again. Adding a new **family** is easy.
- With only one family, the extra interfaces are overhead.

6.9 Related patterns

Each creation method is a **Factory Method**; the factory is often a single instance; **Bridge** (m05) also separates two dimensions of variation.

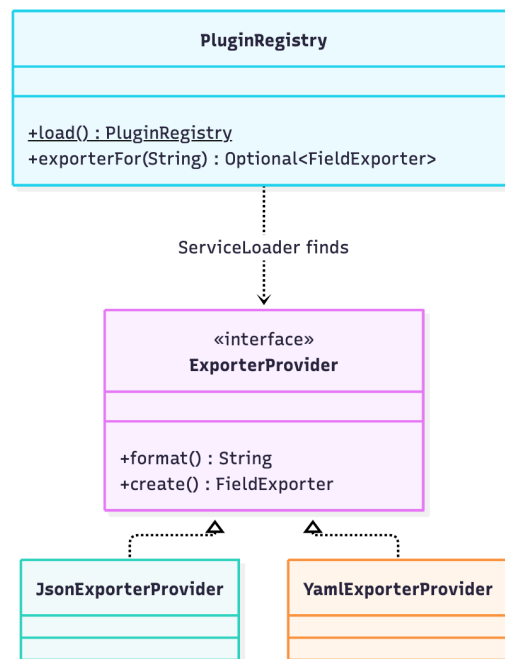
7 ServiceLoader

7.1 Problem

An application should support export formats that are written *after* it ships — as plugins in separate JARs. The code cannot name classes that do not exist yet.

7.2 How it works

1. Define a **service provider interface** (SPI) — here `ExporterProvider`.
2. Each provider implements it and has a public no-argument constructor.
3. The provider's JAR lists it in `META-INF/services/<fully qualified SPI name>`, one class name per line.
4. `ServiceLoader.load(ExporterProvider.class)` finds and instantiates them at run time.



7.3 Example

The configuration file `src/main/java/META-INF/services/io.github.aliturgutbozkurt.patterns.m02.examples.serviceloader.ExporterProvider`

```
# Exporter plugins found by ServiceLoader (see PluginRegistry). One fully qualified
class name per line.
io.github.aliturgutbozkurt.patterns.m02.examples.serviceloader.JsonExporterProvider
io.github.aliturgutbozkurt.patterns.m02.examples.serviceloader.YamlExporterProvider
```

The registry never names a provider class:

```
// file: examples/serviceloader/PluginRegistry.java
/** A registry over every provider {@link ServiceLoader} finds on the class path. */
public static PluginRegistry load() {
    return new PluginRegistry(ServiceLoader.load(ExporterProvider.class).stream()
        .map(ServiceLoader.Provider::get)
        .toList());
}
```

`ServiceLoader.stream()` returns lazy `Provider` handles: `provider.type()` tells you the class **without** instantiating it, `provider.get()` creates it. With the module system the same registration is written in `module-info.java` as `provides ... with ...`.

```
providers on the class path: [JsonExporterProvider, YamlExporterProvider]
formats: [json, yaml]
```

This module keeps the configuration file next to the sources (and copies it for the Maven build), so `java PluginDemo.java` finds the plugins without any build step.

7.4 Real-world usage

JDBC drivers, `java.nio.file.spi.FileSystemProvider` (the zip file system), `javax.script` engines, and `java.time.zone.ZoneRulesProvider` are all discovered with `ServiceLoader`.

7.5 Pitfalls and when NOT to use it

- A typo in the configuration file fails only at run time (`ServiceConfigurationError`).
- Iteration order is not specified — sort, as `PluginRegistry` does, if order matters.
- For code you ship together, a plain factory is simpler and checked by the compiler.

8 Choosing a creational technique

Situation	Use
One obvious class, no variation	<code>new</code> — really
Clearer names, caching, parsing, or choosing a subtype	Static Factory Method
An algorithm in a base class needs a product that varies	Factory Method (or a <code>Supplier</code>)
Several products that must match each other	Abstract Factory
Implementations unknown at compile time (plugins)	<code>ServiceLoader</code>
"There must be only one"	One instance created by the composition root; <code>enum Singleton</code> only for true global constants

9 Summary

Pattern	Use when	Avoid when	Java 27 shortcut
Singleton	A single, stateless or immutable global resource	It hides a dependency or holds mutable state	<code>enum X { INSTANCE }</code>
Static Factory Method	Names, caching or subtype choice help callers	A plain constructor is clear enough	Factory on a sealed interface
Factory Method	A base-class algorithm needs a varying product	The subclass would only call <code>new</code>	<code>Supplier<T></code> , <code>enum of X::new</code>
Abstract Factory	Products must come from one family	There is only one family	Private nested records as products
<code>ServiceLoader</code>	Plugins added after release	Everything ships together	<code>ServiceLoader.stream()</code>

10 Quiz

1. Why can neither serialization nor reflection create a second instance of an `enum` Singleton?
2. What guarantees that the lazy holder idiom is both lazy and thread-safe?
3. Name two concrete problems caused by `SequenceGenerator.getInstance()` inside `OrderService`.
4. Give three things a static factory method can do that a constructor cannot.
5. Why is `Percentage.of` allowed to return shared instances, while a mutable class should not?
6. In the classic exporter, which method is the factory method and which is the template method?
7. When would you prefer an `enum` of constructor references to a subclass per format?
8. What must change when an Abstract Factory gets a new *product*, and when it gets a new *family*?
9. How does `ServiceLoader` find `JsonExporterProvider`, and what happens if the class name in the file is misspelt?

Answers

1. Deserializing an `enum` constant looks up the existing constant by name, and `Constructor.newInstance` refuses to create enum objects (`IllegalArgumentException`).
2. The nested `Holder` class is initialized only when `getInstance()` first reads `Holder.INSTANCE`, and the JVM runs class initialization exactly once, under a lock.
3. The dependency is hidden (not in the constructor), and the global mutable counter makes services and tests interfere with each other.
4. Have a descriptive name, return a cached instance, return a subtype chosen from the input (also: parse input before deciding).
5. `Percentage` is immutable, so sharing is invisible to callers; sharing a mutable object would leak changes between them.
6. `createFormatter()` is the factory method; `export(Report)` is the template method that calls it.
7. When each subclass would do nothing but call one constructor — the `enum` is one line per format and cannot grow other responsibilities.
8. A new product changes the factory interface and every family; a new family is just one new factory class.
9. It reads `META-INF/services/<SPI name>` on the class path and instantiates each listed class; a misspelt name throws `ServiceConfigurationError` at run time.

11 Assignments

- 01 — Colour values with static factories ★★☆
- 02 — Game levels with an Abstract Factory ★★☆

12 Further reading

- Gamma, Helm, Johnson, Vlissides, *Design Patterns* (1994) — Singleton, Factory Method, Abstract Factory.
- Joshua Bloch, *Effective Java*, 3rd ed. (2018), items 1 (static factory methods) and 3 (enum singleton).
- Java Language Specification, §12.4 [Initialization of Classes and Interfaces](#)
- `java.util.ServiceLoader` API documentation
- JEP 531 — [Lazy Constants \(Third Preview\)](#)
- Terms in Turkish: [docs/glossary.md](#)